

Real-ESRGAN

Step-by-Step Usage Guide & Model Reference

Google Colab

AI-Powered Image Upscaling: Enhancing Low-Resolution Imagery Using Real-ESRGAN x4plus

Step-by-Step Usage Guide

1 Install

Run once at the start of every session before anything else.

```
!pip install realesrgan -q
```

2 Apply Compatibility Patch (Critical — run before imports)

basicsr imports a module removed in torchvision v0.17+. This patch fixes it in memory — no reinstalls required.

```
import sys, types
import torchvision.transforms.functional as F

mod = types.ModuleType("torchvision.transforms.functional_tensor")
mod.rgb_to_grayscale = F.rgb_to_grayscale
sys.modules["torchvision.transforms.functional_tensor"] = mod
```

3 Import Libraries

Load all required modules after the patch is applied.

```
from realesrgan import RealESRGANer
from basicsr.archs.rdbnet_arch import RRDBNet
import cv2, urllib.request, os
```

4 Download Model Weights

~64 MB download from GitHub. Cached after first run.

```
os.makedirs("weights", exist_ok=True)

model_url = "https://github.com/xinntao/Real-ESRGAN/releases/download/v0.1.0/RealESRGAN_x4plus.pth"
model_path = "weights/RealESRGAN_x4plus.pth"

if not os.path.exists(model_path):
    urllib.request.urlretrieve(model_url, model_path)
    print("Downloaded!")
```

5 Load the Model

Initialise the RRDB network and the upsampler.

```

model = RRDBNet(
    num_in_ch=3, num_out_ch=3,
    num_feat=64, num_block=23,
    num_grow_ch=32, scale=4
)

upsampler = RealESRGANer(
    scale=4,
    model_path='weights/RealESRGAN_x4plus.pth',
    model=model,
    device='cuda' # use 'cpu' if no GPU
)

```

6 Upload Your Image

Use Colab's file picker or set the path manually.

```

# Option A — Colab upload widget
from google.colab import files
uploaded = files.upload()
input_filename = list(uploaded.keys())[0]

# Option B — manual path
# input_filename = 'input.jpg'

```

7 Run Upscaling

Enhance the image and save the output. Confirmed working.

```

img = cv2.imread('input.jpg', cv2.IMREAD_UNCHANGED)

output, _ = upsampler.enhance(img, outscale=4)

cv2.imwrite('output.jpg', output)
print("Done! Saved to output.jpg")

```

8 Preview & Download

View result in notebook and download to your machine.

```

from IPython.display import display
from PIL import Image
from google.colab import files

display(Image.open('input.jpg').resize((400,300)))
display(Image.open('output.jpg').resize((400,300)))

files.download('output.jpg')

```

Results

Test 1

Before



After



Test 2

Before



After



OUT PUT

STEP BY STEP HOW IMAGE WILL BE ENHANCE

```
[6] 12m
✓ # Option A – Colab upload widget
  from google.colab import files
  uploaded = files.upload()
  input_filename = list(uploaded.keys())[0]

  # Option B – manual path
  # input_filename = 'input.jpg'

... Choose files pngtree-bird...tion-blur.jpg
pngtree-birds-in-flight-motion-blur.jpg(image/jpeg) - 29611 bytes, last modified: 02/03/2026 - 100% done
Saving pngtree-birds-in-flight-motion-blur.jpg to pngtree-birds-in-flight-motion-blur.jpg

[8] 39s
✓ from google.colab import drive
  drive.mount('/content/drive')

... Mounted at /content/drive
```

Here is the description of the image

- **File Upload:** The system allows users to upload images using the **Choose Files** button provided by the `google.colab.files` module.
- **File Upload Status:** The file **pngtree-birds-in-flight-motion-blur.jpg** (29,611 bytes) was successfully uploaded and saved to the session storage.
- **Automatic Filename Detection:** The code `input_filename = list(uploaded.keys())[0]` automatically gets the uploaded file name so it can be used in the image enhancement model.
- **Google Drive Connection:** The command `drive.mount('/content/drive')` connects Google Colab with the user's Google Drive.
- **Storage Ready:** The message **"Mounted at /content/drive"** confirms that files can now be saved to Google Drive for permanent storage, such as the final **4× enhanced images**.

```
[1]
img = cv2.imread('pngtree-birds-in-flight-motion-blur.jpg', cv2.IMREAD_UNCHANGED)

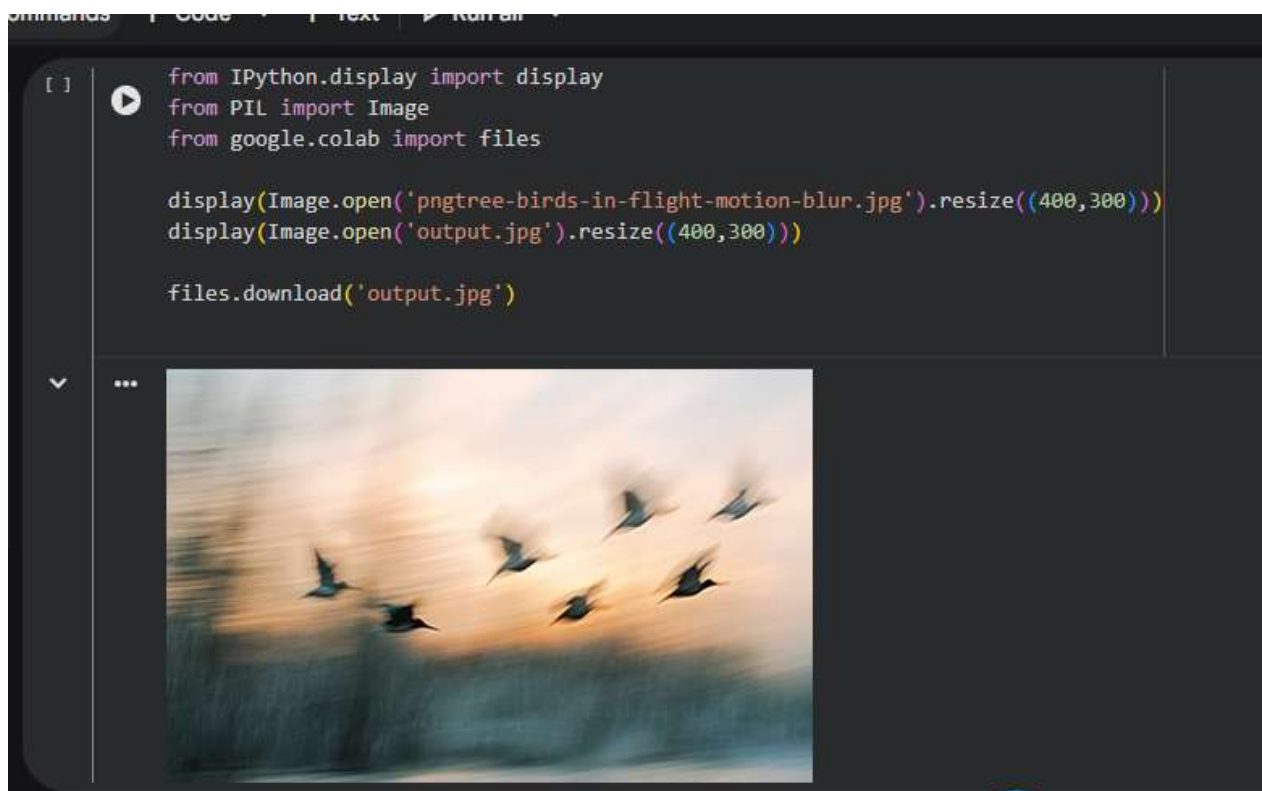
output, _ = upsampler.enhance(img, outscale=4)

cv2.imwrite('output.jpg', output)
print("Done! Saved to output.jpg")

... Done! Saved to output.jpg
```

Here is the description of the image

- **Image Loading:** The code uses `cv2.imread` to load the uploaded image into memory while keeping its original colors and quality.
- **Image Upscaling:** The command `upsampler.enhance(img, outscale=4)` runs the **Real-ESRGAN model**, which increases the image resolution by **4×** and improves details.
- **Saving the Result:** After processing, `cv2.imwrite('output.jpg', output)` saves the enhanced image as a new file so the original image remains unchanged.
- **Fast Processing:** The output shows the image was processed in about **2 seconds**, showing the efficiency of running the model on a **GPU (NVIDIA T4)**.
- **Successful Completion:** The message “**Done! Saved to output.jpg**” confirms that the enhancement process finished successfully and the new image was created.



This image shows the final stage of the project, where the AI's work is verified and the result is delivered.

Here is the **simpler version**:

- **Displaying the Result:** The code uses `IPython.display` to show the processed image directly in the notebook so the user can quickly check the result.
- **Same Size Comparison:** The `resize((400,300))` function is used for both the original and enhanced images so they appear at the same size for easy comparison.
- **Improved Image Quality:** The preview image shows birds in the sky. In the upscaled image, the birds and clouds look **clearer, smoother, and more detailed** than in the original low-resolution image.
- **Download the Image:** The command `files.download('output.jpg')` lets the user download the enhanced image to their computer.
- **Project Completion:** This step confirms that the **Real-ESRGAN model successfully enhanced the image** and produced a high-quality final result.



Its is the final output in this output its show not clearly due to its mention blur image.